ExoNet: Testing Plan

ITAI 2372 Final Project, Implementation Track

Trilok, Group 01 (solo). April 2026.

1. Overview

I am testing this project at three different levels. Unit tests cover the small pieces of the pipeline in isolation. An end-to-end test runs the whole training flow on a tiny synthetic dataset to catch wiring bugs. Then I have a model-validation step where I look at the metrics and confusion matrix on the real KOI data to decide if the model is actually useful.

All automated tests live in the tests/ folder and run with pytest.

2. Unit Tests

These cover individual functions. They use small synthetic inputs so they run fast and do not depend on a network connection.

File	What it covers
test_preprocess.py	preprocess() returns the right shape, fills NaNs, drops CANDIDATE rows, raises on empt
test_train.py	End-to-end train + evaluate on synthetic data. Also checks the train/test split is stratified (
test_predict.py	predict_one() returns a valid label and probability, and raises if a feature is missing.

Run them with:

```
pytest -v
```

3. End-to-End Test

test_end_to_end_synthetic_pipeline in test_train.py is really an integration test, not a unit test. It walks through every step (load synthetic data, preprocess, split, train, save, evaluate) and checks the output files actually appear and the metrics are in the expected ranges. This is the test that catches mismatched function signatures or files written to the wrong path.

4. Model Validation

After running python -m src.main train on the real Kepler data, I look at:

- Accuracy and ROC-AUC. If accuracy drops below 0.85 or ROC-AUC below 0.92, something probably regressed.
- Confusion matrix. If false positives spike, I might need to lower the decision threshold or look at class weights again.

- Feature importances. If something I expected to matter (like koi_prad or koi_model_snr) shows near-zero importance, that is a sign of a preprocessing bug.

I am tracking the latest numbers in data/metrics.json. Current results from the real dataset:

Metric	Value
Test rows	1,518
Accuracy	0.926
Precision	0.883
Recall	0.918
F1	0.900
ROC-AUC	0.976

5. Edge Cases I Test For

- Empty DataFrame passed to preprocess: should raise ValueError, not crash mysteriously later.
- Required columns missing: same, raise early with a clear message.
- Rows with all-NaN features: dropped silently with an INFO log.
- Predict called before train: should give a clear FileNotFoundError pointing at the missing model file.
- Predict called with one feature missing: raises ValueError that lists which feature.

6. Acceptance Criteria

I consider the project ready to submit when all of the following are true:

- All 9 pytest tests pass.
- Real-data accuracy is at least 0.85 and ROC-AUC is at least 0.92.
- python -m src.main train followed by python -m src.main predict ... works on a clean install starting from just the requirements.txt.
- README, proposal, and testing plan are present in the repo and readable as written.

7. Manual Checks Before Submitting

These are not automated, but I plan to walk through them before I submit:

- Delete data/, then run train + predict from scratch. Confirms nothing is hardcoded against my cache.
- Skim the README from a stranger's perspective. Did I assume any context that is not actually in the repo?
- Open the proposal and testing plan PDFs to confirm there are no obvious typos or layout issues.
- Open presentation.pdf and read it slide by slide.
- Confirm the folder is named exactly FP_GROUP01_Trilok_ITAI2372.

8. Things I Am Not Testing

Some things would be nice to test but are out of scope for a single-person project:

- Cross-validation. I am only doing one stratified train/test split. K-fold would give better confidence intervals on the metrics.
- Robustness to schema changes from the NASA archive. If they rename a column, my loader will fail in an obvious way but I do not have a regression test for that.

- Performance under load. This is a CLI used one row at a time, so I am not benchmarking throughput.